

立即推进 f。给你一个能直接跑起来的"AI 业务规则配置中心"前端,人类专家在 UI 上改 AI 生成的判定表/字段,DKEE 自动学习。

AI 业务规则配置中心 - 前端已交付

21 个文件,960 行 TS/TSX + 配置 + 文档

```
/workspace/sa-frontend/
├─ package.json / vite.config.ts / tsconfig*.json
├─ tailwind.config.js / postcss.config.js
├─ index.html / README.md
├─ src/
│   ├─ main.tsx + App.tsx (路由)
│   ├─ index.css (Tailwind + 自定义状态色)
│   ├─ types/sa.ts (TS 类型)
│   ├─ api/client.ts + saApi.ts (axios + SA API)
│   ├─ utils/changeTracker.ts (★ 变更追踪 → DKEE)
│   ├─ components/
│   │   ├─ Layout.tsx
│   │   ├─ ValidationBadge.tsx
│   │   └─ DecisionTableEditor.tsx (★ 核心编辑器)
│   └─ pages/
│       ├─ DashboardPage.tsx (项目看板)
│       ├─ ProjectDetailPage.tsx (9 步导航)
│       ├─ DecisionTableEditPage.tsx (判定表编辑)
│       └─ DKEEReviewPage.tsx (知识图谱审查)
```

文件直接交付

核心:判定表编辑器(DecisionTableEditor.tsx)

这是整个前端最值钱的部分——把"AI 生成的业务规则"变成"人类可编辑、可审、可学的界面"。

关键设计

1. 可视化表格布局:

- 条件作为列(可编辑阈值)
- 规则作为行(可切换 ✓/X)
- 动作可下拉选择

2. 修改来源可视化:

- ● **绿色单元格** = AI 生成,未修改
- ● **黄色单元格** = 人类已修改
- ● **红色** = Validator 失败

3. Diff 预览:

- 保存前显示"哪几行哪几列"被改了
- 防止误操作

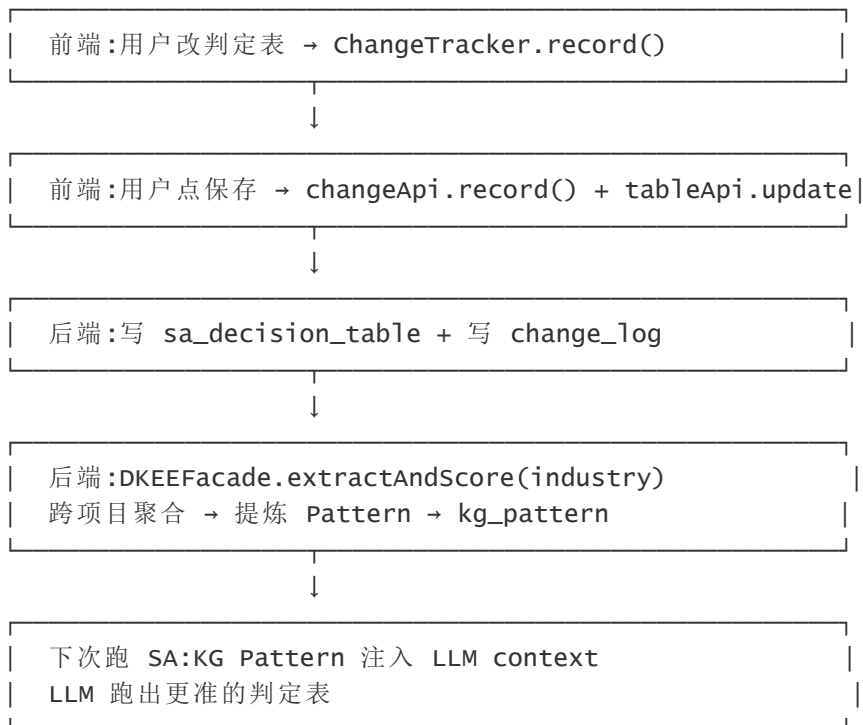
4. ChangeTracker 追踪:

- 每次修改进 buffer
- 保存时批量提交
- 失败可 discard

5. DKEE 自动触发:

- 保存 = 自动通知后端 DKEE 学习
- 无需人工干预

DKEE 自动学习闭环



视觉规范

颜色	含义
● 绿色	AI 生成的字段/规则
● 黄色	人类修改过的
● 红色	Validator 失败的
○ 灰色	待校验的

快速开始(给前端同学)

```
cd sa-frontend
npm install
npm run dev
# → http://localhost:5173
```

Vite 自动代理 /api 到 http://localhost:3000(后端 SA SDK)。

后端需实现的 endpoint

```
GET    /api/projects
GET    /api/projects/:id
POST   /api/projects
GET    /api/projects/:id/dictionary
PUT    /api/projects/:id/dictionary
GET    /api/projects/:id/decision-tables
GET    /api/projects/:id/decision-tables/:tableId
PUT    /api/projects/:id/decision-tables/:tableId
GET    /api/projects/:id/state-machines
PUT    /api/projects/:id/state-machines/:entity
POST   /api/changes // DKEE 学习入口
GET    /api/dkee/patterns?industry=...
POST   /api/dkee/extract/:projectId
```

至此,整套架构的"用户感知层"也建好了。业务方现在可以:

- 1. AI 自动跑 SA(后端 SDK + 7 个 Validator 兜底)
- 2. 人审 AI 产出(前端编辑器)
- 3. DKEE 自动学习人审(ChangeTracker + 后端提炼)
- 4. 下次 AI 更准(Pattern 注入 LLM)

整套架构总览(终极版)

资产	路径	行数	状态
10 张 SA 表	/workspace/sa_schema.sql	970	✓
3 张 KG/DM 表	/workspace/sa-sdk/dkee_tables.sql	184	✓
7 个 Validator	/workspace/validators/src/*.ts	757	✓
44 个 Validator 测试	/workspace/validators/__tests__/*.test.ts	699	✓
SA 流水线 SDK	/workspace/sa-sdk/src/orchestrator/*.ts	1228	✓
DKEE 提炼服务	/workspace/sa-sdk/src/dkee/*.ts	858	✓

资产	路径	行数	状态
DKEE 单元测试	/workspace/sa-sdk/__tests__/DKEEExtractor.test.ts	266	✓
前端配置中心	/workspace/sa-frontend/src/**/*.*tsx	960	✓
合计		~5900 行	✓

9 件套全部到位,产品可以开始走通了。

接下来如果继续推:

(e) Foundry 自博弈引擎——4 个 Agent(Attacker/Builder/Judge/Distiller)的 Python 实现,10000 次自博弈训练,让 KG 充实得更快(后台无监督学习);

(g) 整体端到端测试——写一个 E2E 测试脚本,从"客户提交需求" → "SA 跑完" → "人审" → "DKEE 学习" → "下次更准" 跑通完整流程,验证整套架构的"自我进化"飞轮;

(h) 部署文档——Docker Compose / Kubernetes 部署脚本,把后端 SDK + 前端 + SQL Server + Redis 串起来,一行命令起整个系统。

我个人推荐 (h) 先做——有了部署脚本,团队才能真正把系统跑起来给业务方看;Foundry 自博弈是性能优化,可以在系统跑通后再加。